DATA ENGINEERING · STUDY & INTERVIEW GUIDE

Apache Spark & PySpark Architecture, Internals & Interview Mastery

A complete beginner-to-advanced handbook covering Spark's internal architecture, execution model, PySpark programming, performance tuning, and 100 interview questions — for self-study, revision, and interview preparation.

ARCHITECTURE | DAG & EXECUTION | PYSPARK | PERFORMANCE TUNING | 100 INTERVIEW Q&A | HANDS-ON PROJECTS

Ponugoti Thanush

Data Engineer

Compiled August 2026

CONTENTS

Table of Contents

PART 1 OF 24

Introduction to Big Data & Apache Spark

1.1 What is Big Data?

Big Data refers to datasets that are too large, too fast-moving, or too varied in structure for traditional single-machine tools (like Excel, or a single MySQL server) to store, process, or analyze within a reasonable time. It is commonly described using the **5 V's**:

Property	Meaning	Example
Volume	Sheer size of data	A bank generating 500 GB of transaction logs per day
Velocity	Speed at which data arrives	Credit card swipes streaming in every millisecond
Variety	Different formats — structured, semi-structured, unstructured	JSON logs, CSV files, images, video, sensor data
Veracity	Quality and trustworthiness of data	Duplicate or missing sensor readings
Value	The business insight extracted from data	Fraud detection, personalized recommendations

> **Real-world analogy:** *Think of a single grocery store cashier trying to serve an entire city's population at once. One cashier (one machine) simply cannot process the load — no matter how fast that one cashier works. Big Data tools exist because we needed many cashiers (many machines) working together in parallel.*

1.2 Problems with Traditional Processing

Traditional systems like a single relational database or a single-threaded Python script process data on **one machine, using one CPU (or a few cores) and one disk**. This breaks down when data grows because:

- **Storage limits** — a single disk cannot hold petabytes of data.
- **Memory limits** — RAM on one machine cannot hold data larger than a few hundred GB.
- **Processing time** — scanning a 10 TB file on one machine can take hours or days.
- **Single point of failure** — if that one machine crashes, the entire job is lost.

The solution the industry converged on was **distributed computing**: split the data across many machines (a cluster), and process each piece in parallel, then combine the results.

1.3 Why Hadoop Wasn't Enough

Hadoop (2006) solved distributed storage (HDFS) and distributed processing (MapReduce) — a genuine breakthrough. But MapReduce had real limitations that motivated Spark's creation:

- **Disk-based between every step** — MapReduce writes intermediate results to disk after every Map and every Reduce stage. Disk I/O is orders of magnitude slower than memory, so multi-step pipelines (common in machine learning and iterative algorithms) became extremely slow.
- **Rigid two-stage model** — every problem had to be forced into "Map then Reduce," which was awkward for graph algorithms, iterative machine learning, and interactive queries.
- **No good interactive/ad-hoc query support** — each MapReduce job had significant startup overhead, making quick, exploratory queries painful.
- **Verbose programming model** — writing raw MapReduce Java code for even simple transformations required a lot of boilerplate.

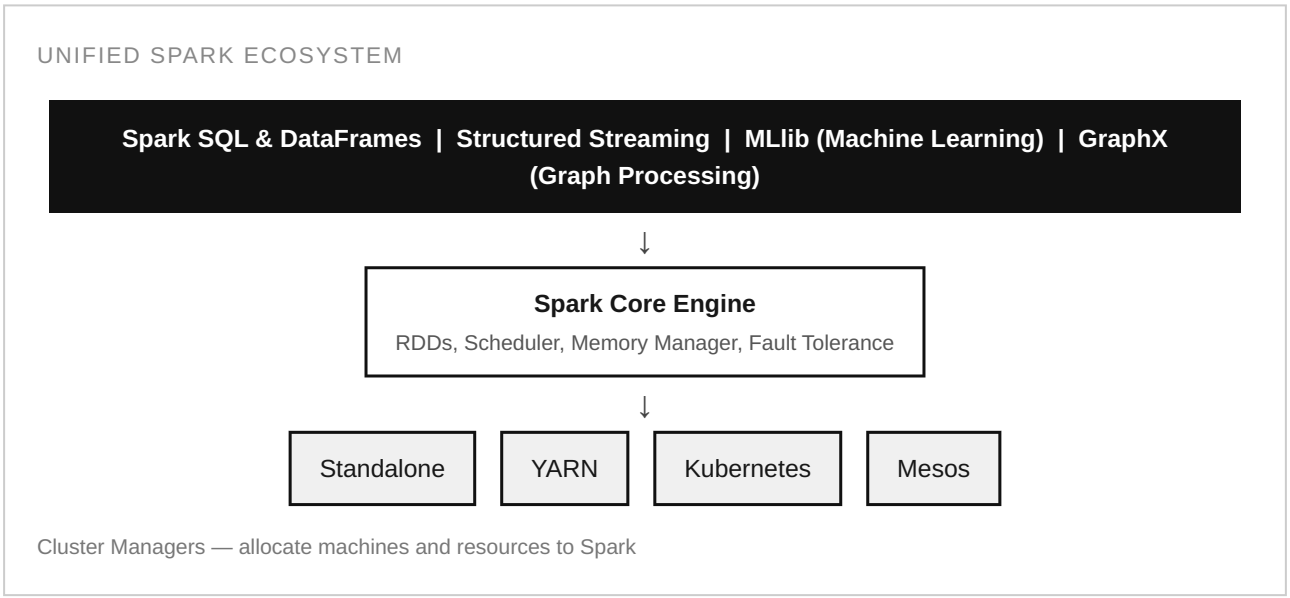
1.4 Why Spark Was Created

Apache Spark was created in 2009 at UC Berkeley's AMPLab (open-sourced in 2010, became an Apache top-level project in 2014) to fix exactly these pain points. Its core idea: **keep data in distributed memory (RAM) across the cluster whenever possible**, instead of writing to disk between every step. This alone made iterative workloads (like machine learning training loops) 10–100x faster than MapReduce in many benchmarks. Spark also introduced a much richer, unified programming model (RDDs, then DataFrames) that supports batch processing, SQL, streaming, machine learning, and graph processing — all within a single engine.

1.5 Evolution of Spark

Year	Milestone
2009	Started as a research project at UC Berkeley AMPLab
2010	Open-sourced under a BSD license
2013	Donated to the Apache Software Foundation
2014	Became an Apache Top-Level Project; Spark 1.0 released; DataFrame API introduced
2016	Spark 2.0 — unified DataFrame/Dataset API, Structured Streaming introduced
2018–2020	Spark 3.0 — Adaptive Query Execution (AQE), Dynamic Partition Pruning, GPU scheduling
2021–Present	Spark 3.x/4.x — Pandas API on Spark, deeper Kubernetes-native support, connect (client-server) architecture

1.6 The Spark Ecosystem



All of Spark's higher-level libraries (SQL, Streaming, MLlib, GraphX) are built on top of the same Spark Core engine, so they share the same optimizations, the same fault-tolerance guarantees, and can be combined freely in one program — e.g., you can read a stream, join it with a static DataFrame, and score it with an MLlib model, all in one job.

1.7 Spark vs Hadoop MapReduce

Aspect	Hadoop MapReduce	Apache Spark
Processing model	Disk-based, two-stage (Map/Reduce)	In-memory, DAG-based (many stages)
Speed	Baseline	Typically 10–100x faster for iterative workloads
Ease of use	Verbose Java API	Concise APIs in Python, Scala, Java, R, SQL
Real-time processing	Not supported natively	Structured Streaming supported
Machine learning	Mahout (separate, limited)	Built-in MLlib
Fault tolerance	Replication-based (HDFS)	Lineage-based (RDD recomputation) + replication

1.8 Spark vs Flink

Aspect	Apache Spark	Apache Flink
Core model	Micro-batch (Structured Streaming) + batch-first design	True native streaming (event-at-a-time), batch is a special case
Latency	Sub-second to a few seconds typically	Sub-millisecond to milliseconds

Maturity / ecosystem	Very large ecosystem, huge community, dominant in batch analytics	Smaller ecosystem, strong in ultra-low-latency streaming use cases
Best fit	Batch ETL, data warehousing, ML pipelines, most enterprise analytics	Real-time fraud detection, complex event processing, low-latency streaming

1.9 Spark vs Hive

Aspect	Apache Hive	Apache Spark (Spark SQL)
What it is	A SQL layer that originally translated queries into MapReduce jobs	A full distributed compute engine with a SQL layer built in
Execution engine	Historically MapReduce (now can use Tez/Spark as engines)	Native Spark engine — DAG, in-memory, Catalyst optimized
Speed	Slower for complex, multi-step queries	Faster due to in-memory execution and cost-based optimization
Use case today	Metadata catalog (Hive Metastore) used by many engines, including Spark	Actual query execution and general-purpose data processing

INTERVIEW TIP — PART 1: A very common opening interview question is "Why is Spark faster than MapReduce?" The strongest answer combines *three* reasons: (1) in-memory computation avoids repeated disk I/O, (2) the DAG scheduler can combine and optimize multiple steps instead of forcing rigid Map→Reduce stages, and (3) the Catalyst optimizer and Tungsten engine generate efficient, low-level executable code. Naming only "in-memory" is a partial/incomplete answer.

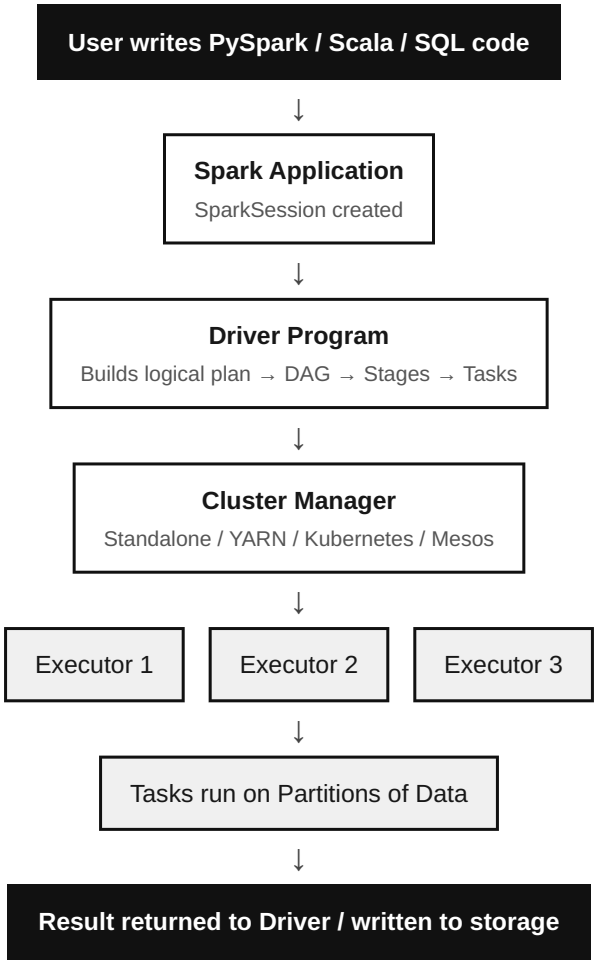
PART 2 OF 24

Apache Spark Architecture

2.1 The Big Picture: What Happens When You Submit a Spark Job?

Spark follows a **master–worker** (also called driver–executor) architecture. When you submit a Spark application, four broad things happen in sequence: (1) a **Driver** process starts and builds a plan of what needs to be done, (2) the driver asks a **Cluster Manager** for resources, (3) the cluster manager launches **Executor** processes on **Worker Nodes**, and (4) the driver breaks the work into small **Tasks** and sends them to executors to run in parallel on partitions of the data.

END-TO-END FLOW — FROM USER CODE TO RESULT



2.2 Every Core Component, Explained

Driver Program

WHAT IT IS

The single process that runs your `main()` function. It owns the `SparkSession` / `SparkContext` and coordinates the entire application.

WHY WE NEED IT

Someone has to convert your high-level code (`df.filter().groupBy()`) into an actual execution plan and hand out work — that is the driver's job.

HOW IT WORKS INTERNALLY

It builds a logical plan, optimizes it via Catalyst, converts it into a physical plan, splits that into a DAG of stages and tasks, and schedules those tasks onto executors. It also tracks task progress and collects final results.

REAL-WORLD ANALOGY

The driver is the construction site **project manager** — reads the blueprint, decides which workers do what, in what order, and tracks completion. The manager does not lay bricks personally.

INTERVIEW EXPLANATION

"The Driver is the brain of a Spark application — it plans, schedules, and coordinates, but the actual data processing happens on executors."

SparkSession & SparkContext

SparkContext is the original entry point to Spark's cluster (connection to the cluster manager, RDD creation). **SparkSession** (introduced in Spark 2.0) is a unified entry point that wraps SparkContext along with SQLContext and HiveContext, giving you one object (`spark`) to create DataFrames, run SQL, and configure the application.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MyFirstApp") \
    .master("local[*]") \
    .getOrCreate()

sc = spark.sparkContext   # SparkContext is still accessible underneath
```

Cluster Manager

WHAT IT IS An external (or built-in) service that owns the pool of machines and decides how much CPU/memory each application gets.
WHY WE NEED IT Spark itself does not own physical machines. It needs a resource broker to negotiate and allocate containers/executors across many competing applications on a shared cluster.
HOW IT WORKS INTERNALLY The driver registers with the cluster manager and requests executors. The cluster manager finds available worker nodes with free resources and launches executor processes on them, then hands connection details back to the driver.
REAL-WORLD ANALOGY Like a **hotel booking system** — you (the driver) ask for 5 rooms (executors) with certain amenities (CPU/RAM), and the booking system finds which hotels (worker nodes) have availability.
INTERVIEW EXPLANATION "Standalone, YARN, Kubernetes, and Mesos are the four cluster managers Spark supports — each is just a different way to obtain and manage executor processes."

Master Node & Worker Node

The **Master Node** runs the cluster manager's master process (e.g., YARN ResourceManager, Spark Standalone Master, or the Kubernetes API server) — it tracks which worker nodes exist and how much capacity each has. **Worker Nodes** are the physical/virtual machines that actually run Executor processes and do the real computation.

Executor

WHAT IT IS

A JVM process launched on a worker node for the lifetime of a Spark application, responsible for running tasks and storing data (cache) in memory or on disk.

WHY WE NEED IT

Actual computation (filtering rows, aggregating, joining) must happen close to the data, in parallel, across many machines — executors are the workhorses that do this.

HOW IT WORKS INTERNALLY

Each executor has a fixed number of CPU cores and a memory budget. It runs one task per core (in parallel threads), reports status back to the driver, and holds cached/persisted partitions in its Block Manager.

REAL-WORLD ANALOGY

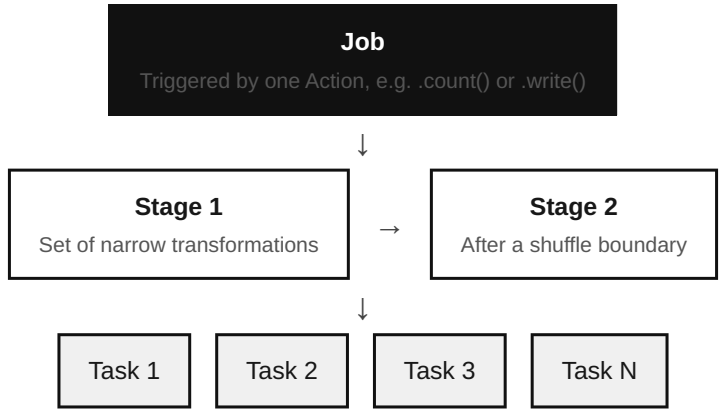
Executors are like **kitchen stations** in a large restaurant — each station (executor) has a few chefs (cores) who each cook one dish (task) at a time.

INTERVIEW EXPLANATION

"Executors live for the life of the application, run many tasks over time, and cache data — unlike tasks, which are short-lived units of work."

Job, Stage, and Task — The Hierarchy

JOB → STAGE → TASK HIERARCHY



One Task runs on exactly one Partition of data, on one executor core

Concept	Definition	Trigger / boundary
Job	Complete unit of work submitted to Spark	Created by every Action (e.g. `count()`, `collect()`, `write()`)
Stage	A set of transformations that can run without a shuffle	New stage starts at every wide transformation / shuffle boundary
Task	The smallest unit of execution	One task per partition, per stage

Partition

A **partition** is a logical chunk of a distributed dataset. Spark splits data into many partitions so that each partition can be processed by a separate task, on a separate core, in parallel. More on this in Part 7.

DAG (Directed Acyclic Graph)

The DAG is Spark's internal map of "what depends on what." Every transformation adds a node to this graph. Because it has no cycles, Spark can safely determine an efficient execution order and recompute only what is necessary on failure. Covered in full detail in Part 4.

DAG Scheduler & Task Scheduler

Component	Responsibility
DAG Scheduler	Converts the logical DAG into physical stages, splitting at shuffle boundaries; tracks which stages depend on which
Task Scheduler	Takes tasks from a ready stage and assigns them to specific executors, considering data locality and available cores; handles retries on task failure

Block Manager

A component inside every executor (and the driver) that manages storage of data blocks — cached RDD/DataFrame partitions, shuffle blocks, and broadcast variables — across memory and disk, and serves them to other executors when needed over the network.

Shuffle Service

Handles the redistribution of data across the cluster required by wide transformations (like `groupBy` or `join`). Covered in full in Part 8. In production clusters, an **External Shuffle Service** can run independently of executors so shuffle data survives even if an executor is dynamically removed.

Memory Manager

Governs how each executor's JVM heap is divided between **storage memory** (cached data) and **execution memory** (shuffles, joins, sorts, aggregations). Covered in full in Part 9.

Catalyst Optimizer & Tungsten Engine

Catalyst is Spark SQL's query optimizer (rewrites your logical plan into an efficient physical plan). Tungsten is the physical execution engine that generates optimized JVM bytecode and manages memory at the binary level. Both are covered in full in Parts 10 and 11.

2.3 How All Components Relate — Colorful Explanation

Picture a Spark application as a **film production**. The **Driver** is the director, holding the full script (your code) and deciding the shot list (the DAG of stages and tasks). The **Cluster Manager** is the production company's resourcing office, who assigns camera crews (executors) from the pool of available freelancers (worker nodes). Each **Executor** is a camera crew with a fixed number of camera operators (**cores**), and each operator can film one scene (**task**) at a time from one location (**partition**). Scenes that

need footage from multiple other scenes first (a **shuffle**, like a big group scene needing footage gathered from everywhere) mark the start of a new **stage**. Once every operator finishes their scenes, the director stitches everything together into the final film (the **result**).

> **INTERVIEW TIP — PART 2:** Interviewers frequently ask you to "explain Spark architecture end-to-end in 2 minutes." Structure your answer as: Driver builds the plan → Cluster Manager allocates Executors → Driver splits work into Stages/Tasks based on shuffle boundaries → Tasks run in parallel on partitions → results are collected or written out. Mentioning the DAG Scheduler and Task Scheduler by name signals deeper understanding.

PART 3 OF 24

Spark Job Execution — A Walkthrough

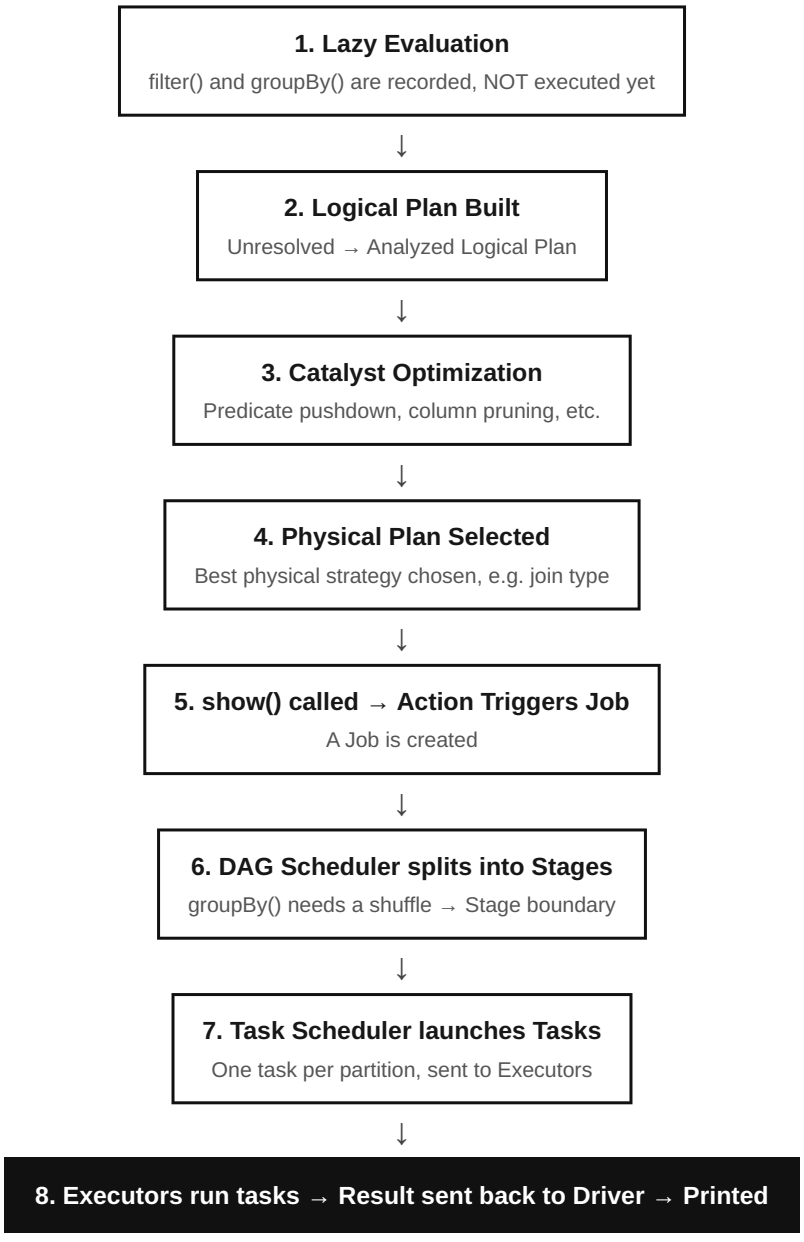
Let's trace exactly what happens internally for one realistic line of code:

```
result = df.filter(df.amount > 1000) \
           .groupBy("customer_id") \
           .agg({"amount": "sum"})

result.show()   # <-- This is the Action that triggers everything
```

3.1 Step-by-Step Internal Execution

FROM CODE TO RESULT



3.2 Lazy Evaluation

`filter()` and `groupBy()` are **transformations** — Spark does not touch any data when these lines run. It merely adds nodes to a logical plan. Only when `show()` (an **action**) is called does Spark actually plan and execute anything. This is **lazy evaluation**, covered fully in Part 12.

3.3 DAG Creation and Optimization

Once an action triggers execution, Spark's Catalyst optimizer inspects the whole chain of transformations together (not one at a time) and rewrites it for efficiency — for example, pushing the `filter(amount > 1000)` down so it happens as early as possible, before the shuffle, so less data needs to move across the network.

3.4 Stage Creation

`filter()` is a **narrow** transformation (no data movement between partitions needed), so it stays in Stage 1. `groupBy().agg()` is a **wide** transformation — rows with the same `customer_id` may live on different machines, so they must be shuffled together. This shuffle boundary forces the creation of Stage 2.

3.5 Task Creation and Execution

If the filtered DataFrame has 200 partitions, Stage 1 creates 200 tasks (one per partition), distributed across all available executor cores. After the shuffle, Stage 2 creates tasks based on the post-shuffle partition count (controlled by `spark.sql.shuffle.partitions`, default 200).

3.6 Result Collection

`show()` only needs the first 20 rows, so Spark is smart enough to avoid computing the entire result where possible; for a full action like `collect()`, every partition's result is sent back to the driver and assembled.

> **INTERVIEW TIP — PART 3:** A favorite scenario question: "If I write 10 transformations followed by one `.count()`, how many jobs, stages, and tasks run?" Answer: exactly **one Job** (one action), the number of **Stages** equals (1 + number of shuffle boundaries), and the number of **Tasks** per stage equals that stage's partition count.

PART 4 OF 24

DAG — Directed Acyclic Graph

4.1 What is a DAG?

A **DAG** is a graph of nodes (operations) connected by directed edges (data dependencies) that contains **no cycles** — you can never follow the arrows and return to where you started. In Spark, every transformation you call adds a node; the edges represent "this operation's output feeds into that operation's input."

4.2 Why Spark Uses a DAG (Instead of MapReduce's Rigid Model)

- It lets Spark see the **entire chain** of operations before running anything, enabling global optimization (versus MapReduce, which optimizes one Map-Reduce pair at a time).
- It enables **precise fault recovery** — if a partition is lost, Spark can recompute only the lost partition by replaying its lineage in the DAG, not the whole job.
- It naturally supports arbitrarily complex pipelines — joins, unions, multiple branches — not just a fixed two-step shape.

4.3 How the DAG Scheduler Works

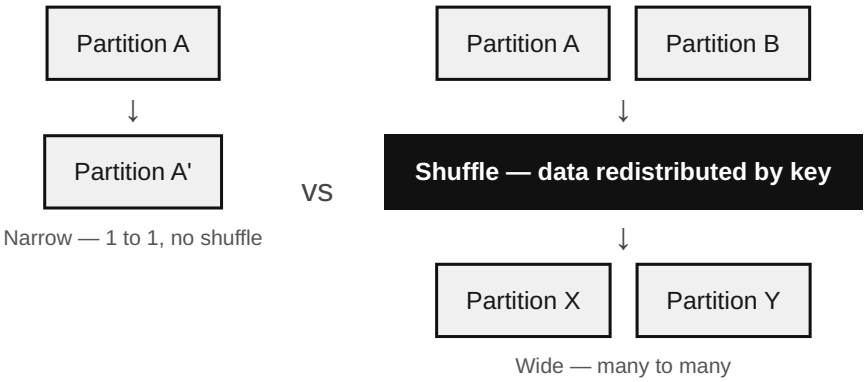
1. An action triggers the DAG Scheduler to look at the final RDD/DataFrame's full lineage.
2. It walks backward through the graph, grouping consecutive narrow transformations into a single stage (this is called **pipelining**).
3. Whenever it hits a wide transformation (needs a shuffle), it "cuts" the graph — this becomes a stage boundary.
4. Stages that don't depend on each other can be scheduled to run concurrently, if resources allow.
5. Each stage is submitted only once all of its parent (upstream) stages have completed.

4.4 Narrow vs Wide Transformations

Aspect	Narrow Transformation	Wide Transformation
Definition	Each input partition contributes to exactly one output partition	Each input partition may contribute to many output partitions
Data movement	None — stays on the same executor	Requires a shuffle across the network
Examples	`map()`, `filter()`, `union()`	`groupBy()`, `join()`, `distinct()`, `repartition()`
Stage impact	Stays within current stage	Creates a new stage boundary

Cost	Cheap	Expensive (network + disk I/O)

NARROW VS WIDE TRANSFORMATION



INTERVIEW TIP — PART 4: "Is `reduceByKey` narrow or wide?" — It is wide (it shuffles), but it is more efficient than `groupByKey` because it performs partial aggregation (a "map-side combine") on each partition *before* shuffling, reducing the amount of data moved across the network.

PART 5 OF 24

Spark Cluster Managers

A cluster manager is pluggable — the same Spark application code runs unchanged regardless of which one you use.

5.1 Standalone

Spark's own simple, built-in cluster manager. A Master process and Worker processes are started directly with Spark's scripts — no external dependency.

Advantages	Disadvantages
Very simple to set up; no extra install	Fewer advanced scheduling/multi-tenancy features
Low overhead for small/dedicated clusters	Not ideal for sharing a cluster across many different frameworks

When companies use it: small teams, dedicated Spark-only clusters, learning/testing environments.

5.2 YARN (Yet Another Resource Negotiator)

Hadoop's resource manager. Very widely used in enterprises that already run a Hadoop/HDFS ecosystem.

Advantages	Disadvantages
Mature, battle-tested, integrates tightly with HDFS and Hive	Heavier setup; tied to the Hadoop ecosystem
Strong multi-tenant resource sharing (queues, capacity scheduler)	Slower container startup vs Kubernetes in cloud-native setups

When companies use it: traditional on-prem Hadoop data lakes, large enterprises (banks, telecoms) with existing Hadoop investments.

5.3 Kubernetes

Runs each executor as a Kubernetes pod. The modern, cloud-native choice.

Advantages	Disadvantages
Cloud-native, elastic autoscaling, container isolation	Requires Kubernetes expertise; shuffle service setup needs care
Same orchestration platform as the rest of a modern microservices stack	Historically less mature dynamic allocation than YARN (improving fast)

When companies use it: cloud-first companies, teams standardizing all workloads (not just Spark) on Kubernetes.

5.4 Mesos

A general-purpose cluster resource manager (largely superseded today by Kubernetes and YARN in new deployments, but still found in some legacy environments).

Advantages	Disadvantages
Fine-grained resource sharing across many frameworks	Declining community support and adoption

5.5 Comparison Table

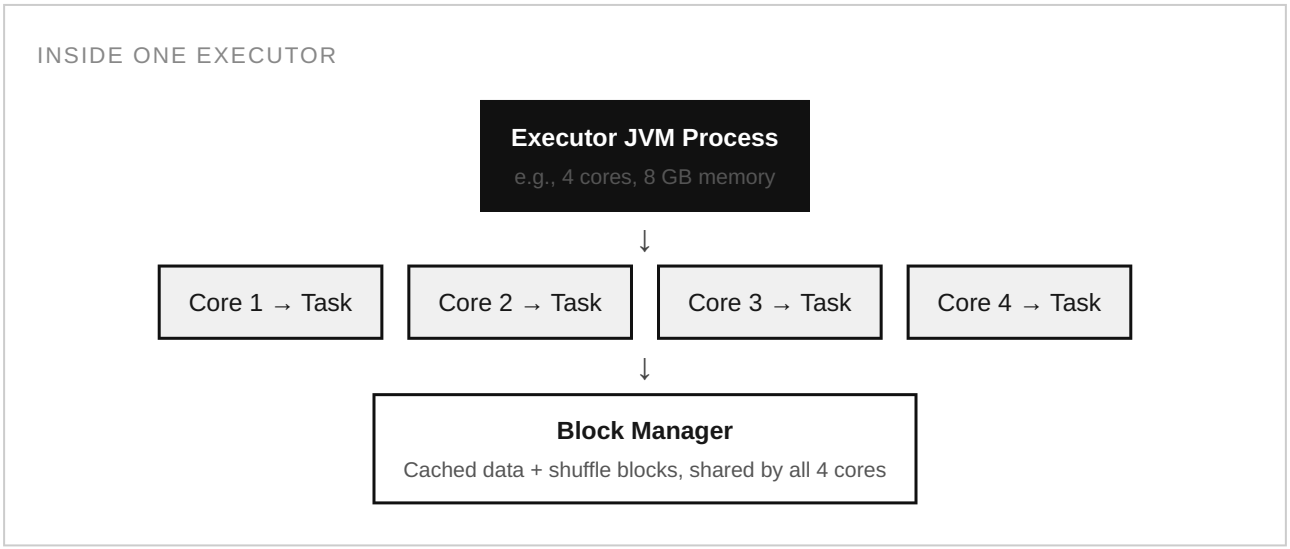
Feature	Standalone	YARN	Kubernetes	Mesos
Setup complexity	Low	Medium–High	Medium	Medium–High
Best for	Small/dedicated clusters	Hadoop-based enterprises	Cloud-native, containerized platforms	Legacy multi-framework clusters
Multi-tenancy	Basic	Strong (queues)	Strong (namespaces, quotas)	Strong
Industry trend	Stable, niche	Still dominant on-prem	Fast-growing	Declining

PART 6 OF 24

Executors

6.1 Executor, Core, Thread, and Task

An **Executor** is a JVM process. Each executor is assigned a number of **cores** (e.g., `--executor-cores 4`). Internally, each core maps to one worker **thread** inside the executor's thread pool, and each thread runs one **task** at a time. So an executor with 4 cores can run 4 tasks concurrently.



6.2 Executor Memory Split

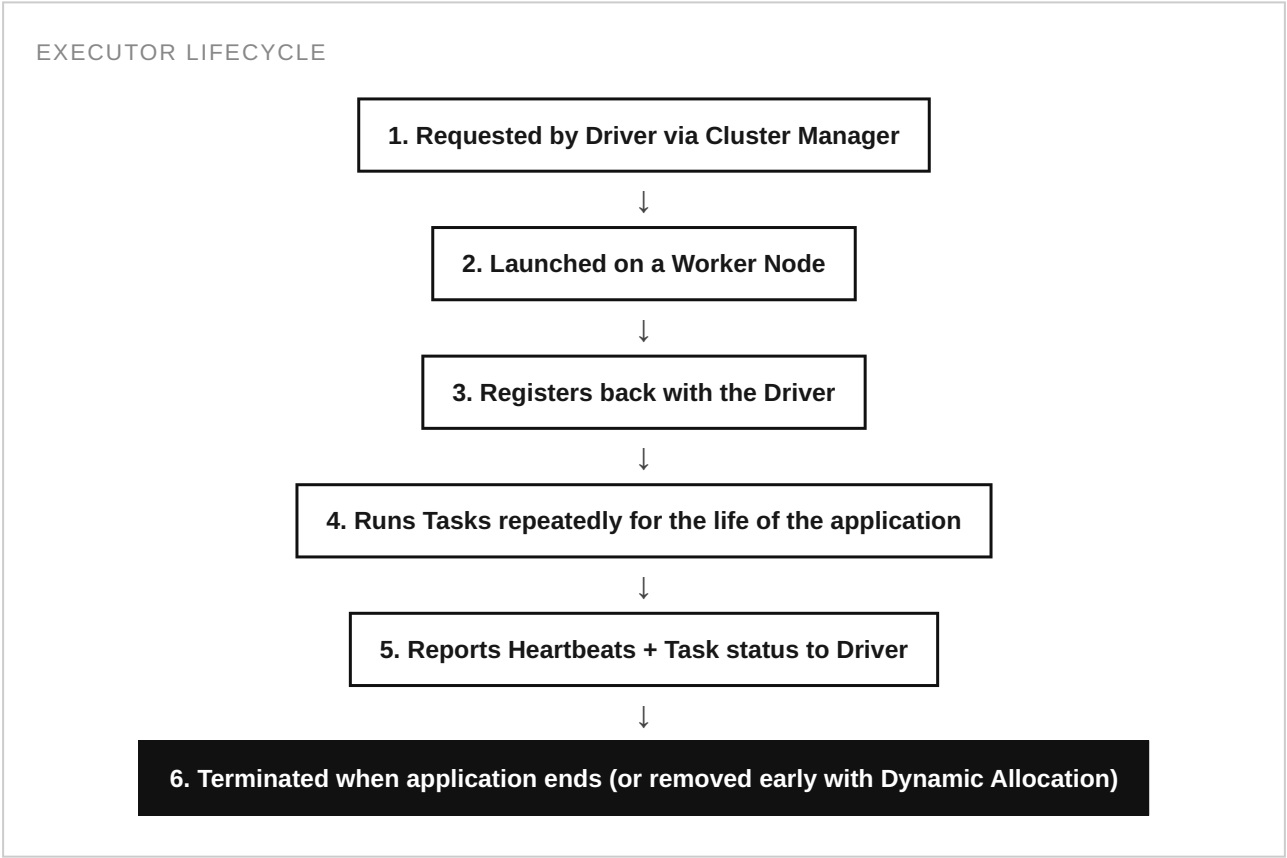
Memory region	Purpose
Storage Memory	Cached / persisted DataFrames and RDDs
Execution Memory	Shuffles, joins, sorts, aggregations (temporary working memory)
User Memory	User-defined data structures, UDF internals
Reserved Memory	Fixed amount reserved for Spark's own internal objects

Storage and Execution memory share a **unified region** that can borrow from each other dynamically (see Part 9).

6.3 Garbage Collection (GC)

Because executors are JVM processes, they are subject to Java garbage collection. Long GC pauses are a common real-world performance problem — too many small objects (common with RDDs of Python/Java objects) can trigger frequent, expensive "stop-the-world" GC pauses that freeze task execution. Using DataFrames (Tungsten's binary format) instead of RDDs greatly reduces this problem because Tungsten stores data off-heap in a compact binary format.

6.4 Executor Lifecycle



INTERVIEW TIP — PART 6: "What's the difference between an Executor and a Task?" — An Executor is a long-lived process (lives for the whole application); a Task is short-lived (runs once, for one partition, then finishes). One executor runs many tasks, sequentially or in parallel, over its lifetime.

PART 7 OF 24

Partitions

7.1 Why Partitions Exist

Partitions are how Spark achieves parallelism. A dataset is split into many partitions so that many tasks (one per partition) can run at the same time across many cores/machines, instead of one core reading the entire dataset sequentially.

> **Real-world analogy:** *Splitting a 1,000-page book among 10 readers, one 100-page chunk each, so all readers finish in the time it takes to read 100 pages, not 1,000.*

7.2 Default Partition Calculation

Source	Default partition count
Reading a file (e.g. CSV/Parquet on HDFS/S3)	Roughly based on input split size (often 128 MB blocks)
`spark.range()` / parallelized collections	`spark.default.parallelism` (usually = total cores available)
After a shuffle (`groupBy` , `join`)	`spark.sql.shuffle.partitions` (default 200)

7.3 repartition() vs coalesce()

Aspect	`repartition(n)`	`coalesce(n)`
Can increase partitions?	Yes	No (only decreases)
Shuffle?	Full shuffle — data evenly redistributed	No full shuffle — merges nearby partitions
Cost	Higher (network shuffle)	Lower (mostly local merge)
Use case	Increasing parallelism, fixing skew, before a wide operation	Reducing output file count before a write, after filtering out most rows

```
# Increase partitions for more parallelism (full shuffle)
df = df.repartition(200)

# Reduce partitions cheaply before writing output files
df = df.coalesce(10)
```

7.4 Partitioning Strategies & Data Skew

Data skew happens when one or a few partition keys hold vastly more data than others (e.g., 90% of transactions belong to "customer_id = NULL" or one mega-customer). The task handling that partition becomes a bottleneck — everyone else finishes in seconds while one task runs for an hour. Common

fixes: **salting** the skewed key with a random suffix to spread it across more partitions, using Adaptive Query Execution's automatic skew join handling (Spark 3+), or broadcasting the smaller side of a join.

7.5 Data Locality

Spark tries to schedule a task on the same node (or rack) where its partition's data already lives, to avoid network transfer. Locality levels, from best to worst: `PROCESS_LOCAL` > `NODE_LOCAL` > `RACK_LOCAL` > `ANY`.

> **INTERVIEW TIP — PART 7:** "Why does my Spark job have so many small output files?" — Usually caused by too many partitions right before a write (e.g., after a wide shuffle with the default 200 partitions on a small result). Fix with `coalesce()` before writing.

Shuffle

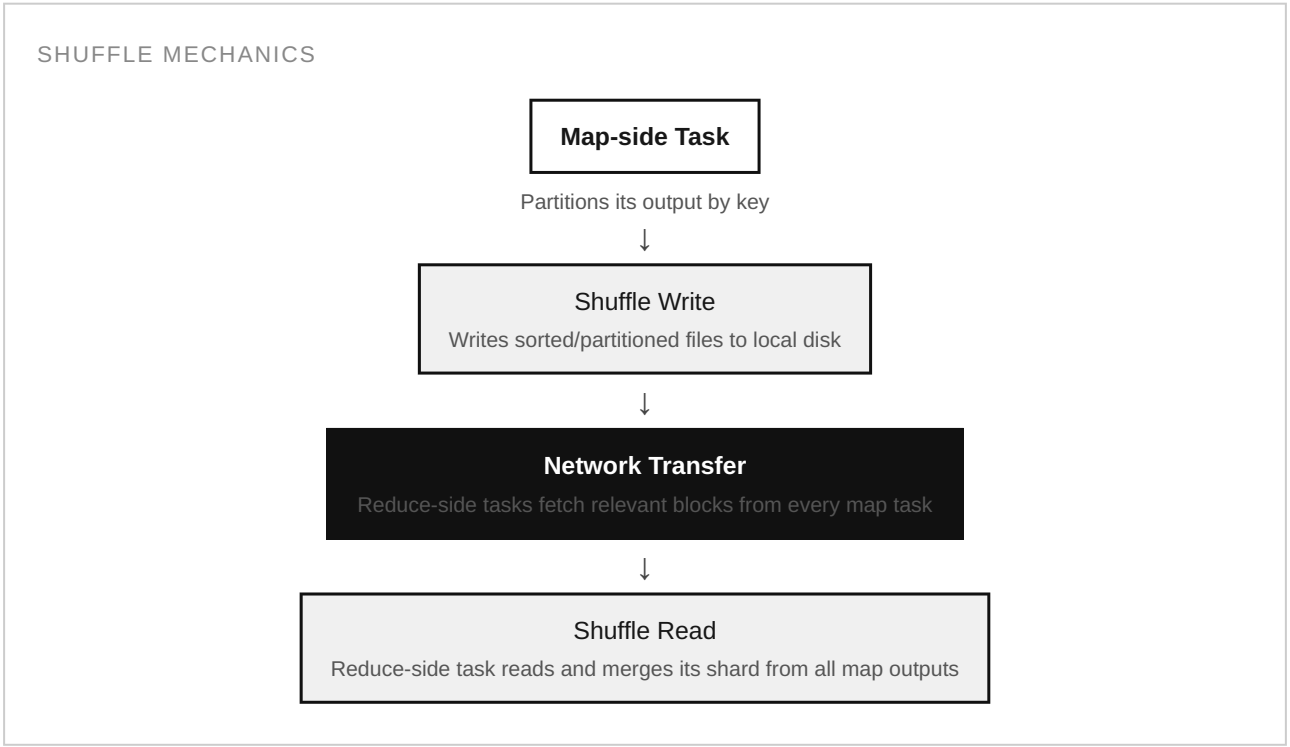
8.1 What is Shuffle?

A **shuffle** is the process of redistributing data across partitions (and across the network, between executors) so that rows sharing the same key end up on the same partition. It is required whenever an operation needs to combine data that may currently live on different machines.

8.2 When Shuffle Occurs

- `groupBy()`, `reduceByKey()`, `aggregateByKey()`
- `join()` (unless one side is small enough to broadcast)
- `distinct()`, `repartition()`
- `sortBy()` / global `orderBy()`

8.3 Shuffle Write and Shuffle Read



8.4 Why Shuffle is Expensive

Cost driver	Explanation
Disk I/O	Shuffle write persists intermediate data to local disk (for fault tolerance)

Network I/O	Every reduce task may fetch data from every map task — an all-to-all transfer
Serialization	Data must be serialized to send over the network and deserialized on arrival
Sorting	Many shuffle implementations sort data by key on both sides

8.5 Shuffle Optimization Techniques

- **Broadcast joins** — if one side of a join is small (fits in memory), broadcast it to every executor to avoid shuffling the large side entirely.
- **Reduce shuffle partitions appropriately** — tune `spark.sql.shuffle.partitions` instead of leaving the default 200 for both tiny and huge datasets.
- **Enable Adaptive Query Execution (AQE)** — Spark 3+ can dynamically coalesce shuffle partitions and handle skewed joins automatically at runtime.
- **Pre-aggregate before shuffling** — prefer `reduceByKey`-style operations that combine locally first, over operations that ship all raw records.
- **Bucketing** — pre-shuffle data once at write time so repeated joins on the same key avoid a shuffle later.

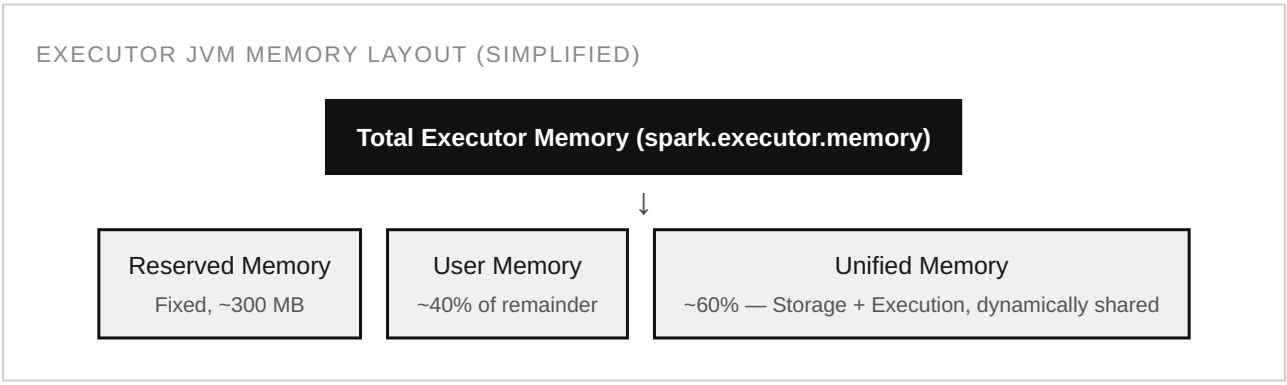
> **INTERVIEW TIP — PART 8:** "How would you optimize a slow join in Spark?" — Check the size of both sides first. If one is small, force/allow a broadcast join. If both are large, ensure the shuffle partition count is reasonable for the data volume, check for skew, and consider bucketing if the join is repeated often.

PART 9 OF 24

Memory Management

9.1 Unified Memory Management (Spark 1.6+)

Modern Spark uses a **Unified Memory Manager**: Storage Memory and Execution Memory share one region and can borrow space from each other dynamically, instead of each having a rigid, fixed boundary (as in older Spark versions). If a task needs execution memory and storage memory has spare capacity, it can evict cached blocks to reclaim space — with execution generally able to evict storage, but not vice-versa beyond a configured minimum.



9.2 Caching and Persistence

`cache()` is shorthand for `persist(MEMORY_AND_DISK)` (or MEMORY_ONLY for RDDs, in some versions). `persist()` lets you choose a storage level explicitly.

Storage Level	Behavior
MEMORY_ONLY	Store as deserialized Java objects in RAM; recompute if it doesn't fit
MEMORY_AND_DISK	Spill to local disk if it doesn't fit in RAM (default for `cache()` on DataFrames)
MEMORY_ONLY_SER	Store serialized (more compact, more CPU to access)
DISK_ONLY	Store only on disk
OFF_HEAP	Store outside the JVM heap (via Tungsten), reduces GC pressure

```
df.cache()                    # shorthand persist
df.persist(StorageLevel.MEMORY_AND_DISK_SER)
df.unpersist()                # release when no longer needed
```

9.3 Spill to Disk

When execution memory (for a sort, shuffle, or aggregation) exceeds what's available, Spark **spills** intermediate data to local disk rather than failing. This keeps jobs running but adds significant disk I/O

overhead — frequent spilling in the Spark UI is a strong signal that a job needs more memory, better partitioning, or reduced data skew.

9.4 Serialization: Java vs Kryo

Serializer	Speed	Size	Notes
Java Serialization	Slower	Larger	Default; works for any class without extra setup
Kryo Serialization	Faster (often 2–10x)	Smaller	Recommended for production; may need classes registered for best results

```
spark.conf.set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
```

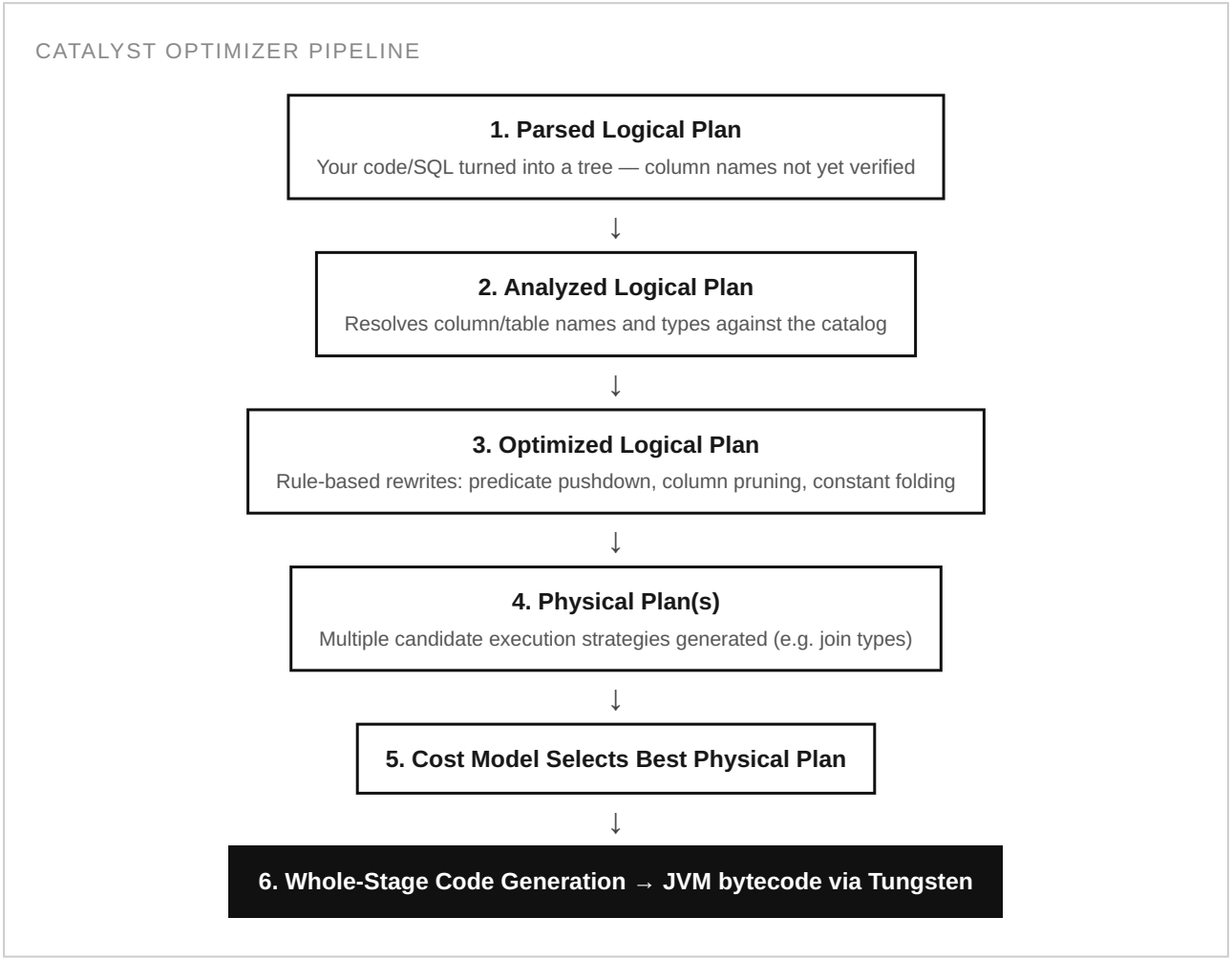
INTERVIEW TIP — PART 9: "My job is spilling to disk — what do you check first?" — Executor memory configuration, shuffle partition count relative to data size, and whether there's data skew concentrating too much data onto too few partitions/tasks.

Catalyst Optimizer

10.1 What is Catalyst?

Catalyst is Spark SQL's query optimizer — a rule-and-cost-based system that transforms the query you write (DataFrame code or SQL) into an efficient execution plan, automatically, without you needing to hand-tune it.

10.2 The Four Phases



Optimization	What it does
Predicate Pushdown	Moves `WHERE` filters as close to the data source as possible (even into the file reader)
Column Pruning	Only reads columns actually referenced downstream, skipping unused ones (huge win for Parquet)
Constant Folding	Pre-computes constant expressions like `1 + 2` at plan time

Join Reordering	Reorders multi-way joins to minimize intermediate data size

> **Real-world analogy:** *Catalyst is like a GPS route planner — you tell it your destination (the result you want), and it independently figures out the fastest route (execution plan), considering current traffic (data statistics), rather than you manually planning turn-by-turn directions.*

INTERVIEW TIP — PART 10: "Does Catalyst optimize RDD code?" — No. Catalyst only optimizes DataFrame/Dataset/SQL code, because it needs a structured plan with schema information. Raw RDD transformations are opaque, black-box functions to Spark, so they cannot be optimized this way — this is a major reason DataFrames outperform RDDs.

PART 11 OF 24

Tungsten Engine

11.1 Why Tungsten Exists

Even with a perfect query plan from Catalyst, actually *executing* that plan efficiently on real CPUs and memory required deeper, lower-level engineering. Project Tungsten (introduced in Spark 1.4+) focuses on getting Spark closer to "bare metal" performance by improving CPU and memory efficiency directly, rather than relying purely on the JVM's default object model.

11.2 Memory Optimization

Tungsten stores data in a compact **binary row format** off the JVM heap (or in a very compact on-heap representation), rather than as regular Java objects. Java objects carry significant memory overhead per object (object headers, pointers, boxing); Tungsten's binary format avoids nearly all of that, and drastically reduces garbage collection pressure.

11.3 CPU Optimization & Whole-Stage Code Generation

Normally, executing a chain of operators (filter, then project, then aggregate) means calling a function per-row, per-operator — lots of virtual function call overhead. **Whole-Stage Code Generation (WSCG)** collapses an entire stage's chain of operators into a single, tight, auto-generated Java function, similar to what a human would hand-write, eliminating the overhead of per-operator virtual dispatch.

> **Real-world analogy:** *Instead of an assembly line where a part is picked up and put down by a different worker at each of 5 stations (overhead every handoff), Whole-Stage Code Generation is like one highly efficient worker doing all 5 steps on the part without ever putting it down.*

INTERVIEW TIP — PART 11: "What's the relationship between Catalyst and Tungsten?" — Catalyst decides *what* to do (the plan); Tungsten decides *how to run it fast* at the CPU/memory level (binary format + generated code). They work together but solve different problems.

Lazy Evaluation

12.1 The Core Idea

Transformations (`select`, `filter`, `join`, `groupBy`...) do not execute immediately — Spark only records *what* you want done. Execution begins only when an action (`show`, `collect`, `count`, `write`...) is called.

12.2 Execution Timeline Example

```
df1 = spark.read.csv("sales.csv")     # Nothing executes — just defines a plan
df2 = df1.filter(df1.amount > 100)     # Still nothing executes
df3 = df2.select("customer", "amount")  # Still nothing executes
df3.count()                            # <-- NOW Spark plans and runs everything, in one pass
```

Time	Line	What actually happens
t0	`read.csv`	Plan node added — no data read from disk yet
t1	`filter`	Plan node added — no rows filtered yet
t2	`select`	Plan node added — no columns selected yet
t3	`count()`	Entire plan optimized as one unit, then executed

12.3 Why Lazy Evaluation Matters

- **Whole-plan optimization** — Catalyst sees the complete chain and can reorder/fuse steps (e.g., push the filter before the read).
- **Avoids wasted work** — if you never call an action, Spark never touches the data at all.
- **Enables pipelining** — multiple narrow transformations can run in a single pass over each partition, without materializing intermediate results.

> **INTERVIEW TIP — PART 12:** "If I write a bug in a transformation, when will I see the error?" — Often only when an action runs, not when the transformation line executes (though schema-related errors on DataFrames may surface earlier during analysis, since Catalyst validates column references at plan-build time).

Transformations vs Actions

13.1 The Core Distinction

Transformations	Actions
Lazy — build the plan only	Eager — trigger execution of the plan
Return a new DataFrame/RDD	Return a value, or write data, to somewhere other than a DataFrame
e.g. `filter` , `map` , `join`	e.g. `count` , `collect` , `show` , `write`

13.2 Common Transformations

Transformation	Type	Purpose
`map()`	Narrow	Apply a function to every element, 1-to-1
`flatMap()`	Narrow	Like map, but flattens nested results (1-to-many)
`filter()` / `where()`	Narrow	Keep rows matching a condition
`distinct()`	Wide	Remove duplicate rows across the whole dataset
`join()`	Wide (unless broadcast)	Combine two datasets on a key
`union()`	Narrow	Stack two datasets with the same schema
`groupBy()`	Wide	Group rows by key for aggregation
`agg()`	Depends	Compute aggregate functions (sum, avg, count...)
`sort()` / `orderBy()`	Wide	Globally sort rows
`select()`	Narrow	Choose/derive columns
`withColumn()`	Narrow	Add or replace a column
`drop()`	Narrow	Remove a column
`dropDuplicates()`	Wide	Remove duplicates, optionally by subset of columns
`repartition()`	Wide	Change the number of partitions (full shuffle)
`coalesce()`	Narrow-ish	Reduce partitions without a full shuffle

13.3 Common Actions

Action	Purpose	Caution
`show()`	Print first N rows (default 20) to console	Safe — bounded output

`collect()`	Bring the entire result to the driver as a local list	Dangerous on large data — can crash the driver (OOM)
`count()`	Return the total row count	Triggers a full scan (unless metadata is cached)
`take(n)`	Return the first n rows to the driver	Safer than `collect()`
`first()`	Return the first row	Equivalent to `take(1)`
`foreach()`	Apply a function to each row (side effects, e.g. write to external system)	Runs on executors, not the driver
`save()` / `write()`	Persist result to storage (Parquet, CSV, database...)	Most common production action

INTERVIEW TIP — PART 13: "Why is `collect()` dangerous?" — It pulls every row from every executor back into the driver's single JVM memory space. On a large dataset this can easily exceed driver memory and crash the whole application — `take(n)`, `write()`, or aggregating on executors first are safer alternatives.

PySpark Introduction

14.1 What is PySpark?

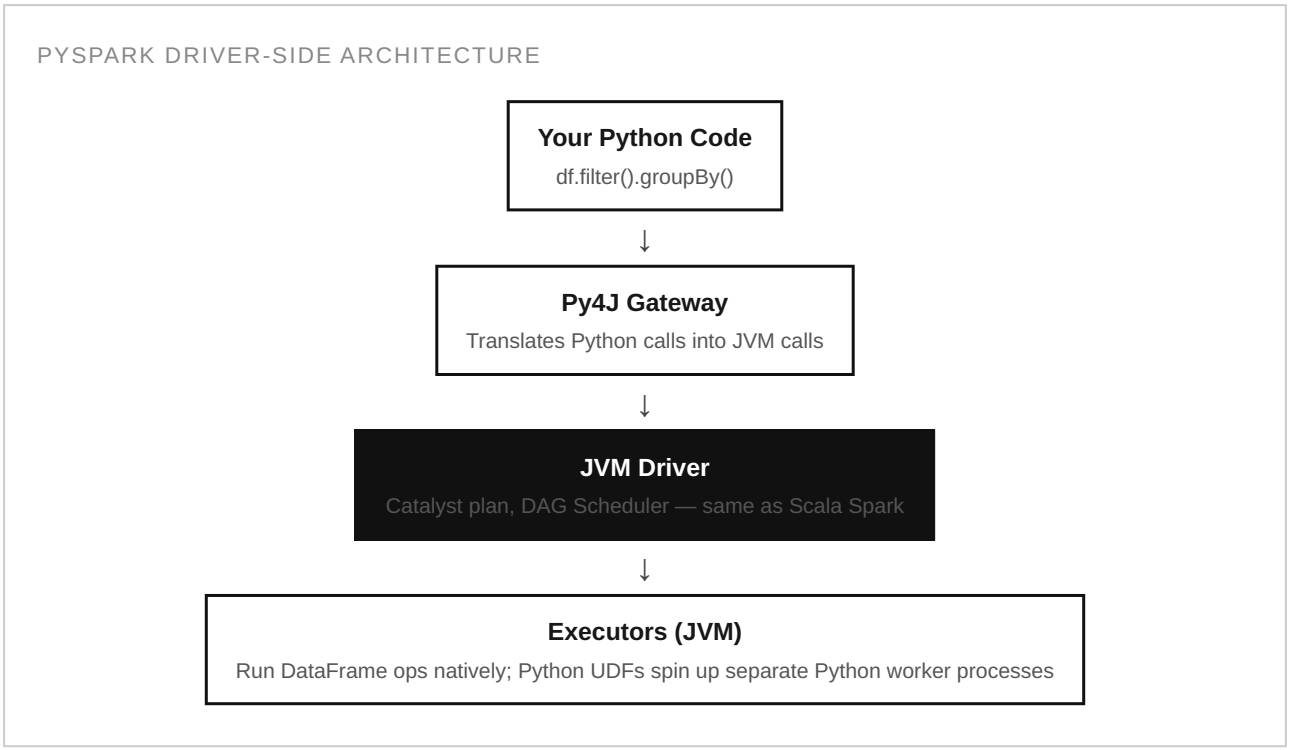
PySpark is the official Python API for Apache Spark. It lets Python developers write Spark applications using familiar Python syntax, while the actual distributed execution still happens on Spark's Scala/JVM engine underneath.

14.2 Why PySpark?

- Python is the dominant language in data analytics, data science, and machine learning — PySpark lets that ecosystem scale to big data.
- DataFrame operations in PySpark run through Catalyst/Tungsten just like Scala — for DataFrame code, there is little to no performance penalty versus Scala.
- Rich integration with pandas, NumPy, scikit-learn workflows.

14.3 How PySpark Communicates with the JVM — Py4J

Python and the JVM are different runtimes. PySpark bridges them using **Py4J**, a library that lets Python code make calls into Java objects running in a JVM process, and vice versa. When you call `df.filter(...)` in Python, PySpark translates that into the equivalent JVM Catalyst plan calls via Py4J — the DataFrame's actual execution plan lives and runs on the JVM side, not in Python.



Important caveat: plain Python **UDFs** (user-defined functions) break this efficiency — each row must be serialized from the JVM, sent to a separate Python worker process, executed in Python, and serialized back. This round-trip is slow. Pandas UDFs (vectorized, via Apache Arrow) reduce this cost significantly.

14.4 SparkSession, DataFrame, RDD, DataFrame API, Spark SQL — How They Relate

Concept	Role
SparkSession	Your single entry point to everything below
RDD	Lowest-level distributed collection API (no schema, no Catalyst optimization)
DataFrame	Distributed table with a schema, built on RDDs internally, optimized by Catalyst
DataFrame API	The Python/Scala method-chaining syntax (`.filter()` , `.select()` ...)
Spark SQL	Write actual SQL strings against DataFrames registered as views — same optimizer, same engine

RDD — Resilient Distributed Dataset

15.1 What is an RDD?

An RDD is Spark's original core abstraction: an **immutable**, partitioned collection of objects, distributed across the cluster, that can be operated on in parallel. "Resilient" refers to its fault-tolerance mechanism.

15.2 Immutability

Once created, an RDD's data never changes. Every transformation (`map`, `filter`...) produces a brand-new RDD rather than modifying the original in place. This makes reasoning about distributed data much simpler and safer — no partition can be corrupted by a concurrent in-place update from another task.

15.3 Fault Tolerance via Lineage

Instead of replicating every RDD's data for safety (expensive), Spark remembers the **lineage** — the exact sequence of transformations used to build it from the original source data. If a partition is lost (e.g., an executor crashes), Spark simply recomputes just that partition by replaying its lineage, rather than restarting the whole job.

> **Real-world analogy:** *Lineage is like a cooking recipe rather than the finished dish. If you drop and lose one plate (partition) at a dinner party, you don't need to re-cook the entire meal — you just look at the recipe and remake that one plate.*

15.4 Advantages and Disadvantages

Advantages	Disadvantages
Full control over low-level data (arbitrary Python/ Java/Scala objects)	No schema — Spark can't reason about columns/types
Good for unstructured data or custom, complex logic	No Catalyst optimization — transformations are opaque black-box functions
Fine-grained transformation control	Higher memory overhead (regular JVM/Python objects, more GC pressure)
Type-safety (in Scala)	More verbose than DataFrame code for common operations

15.5 A Simple RDD Example

```
rdd = sc.textFile("logs.txt")
errors = rdd.filter(lambda line: "ERROR" in line)
error_count = errors.count()   # Action triggers execution
```

INTERVIEW TIP — PART 15: "When would you still use RDDs today, given DataFrames exist?" — For unstructured data with no natural tabular schema, for very fine-grained custom partitioning/control, or for certain legacy code — but for the vast majority of modern data engineering work, DataFrames are strongly preferred due to Catalyst/Tungsten optimization.

DataFrame

16.1 Schema, Columns, Rows

A DataFrame is a distributed collection of rows organized into named, typed **columns** — conceptually like a relational database table or a pandas DataFrame, but distributed across a cluster. The **schema** (column names + types) is known to Spark ahead of execution, which is precisely what unlocks Catalyst's optimizations.

```
df.printSchema()
# root
#  |-- customer_id: string (nullable = true)
#  |-- amount: double (nullable = true)
#  |-- txn_date: date (nullable = true)
```

16.2 Why DataFrames Outperform RDDs

Reason	Explanation
Catalyst optimization	Because the schema is known, Catalyst can rewrite and optimize the plan (RDDs cannot be optimized this way)
Tungsten binary format	Rows are stored compactly off-heap, reducing memory footprint and GC pauses
Whole-stage code generation	Generates fast, fused JVM bytecode for chains of DataFrame operations
Language parity	Python DataFrame code runs at essentially the same speed as Scala, since the plan executes on the JVM either way

16.3 DataFrame Example

```
df = spark.read.option("header", True).csv("transactions.csv")

result = (
    df.filter(df.amount > 1000)
      .groupBy("customer_id")
      .sum("amount")
      .withColumnRenamed("sum(amount)", "total_spent")
      .orderBy("total_spent", ascending=False)
)
result.show(10)
```

PART 17 OF 24

Spark SQL

17.1 Temporary View vs Global Temp View

View type	Scope	Lifetime
Temporary View	Current SparkSession only	Ends when the session ends
Global Temp View	Shared across all sessions in the same Spark application	Stored in the special `global_temp` database; ends when the application ends

17.2 Running SQL Queries

```
df.createOrReplaceTempView("transactions")

result = spark.sql("""
    SELECT customer_id, SUM(amount) AS total_spent
    FROM transactions
    WHERE amount > 1000
    GROUP BY customer_id
    ORDER BY total_spent DESC
""")
result.show(10)
```

17.3 Execution — Same Engine as DataFrame Code

This is a key interview point: **Spark SQL strings and DataFrame method-chain code compile down to the exact same logical/physical plan and the same Catalyst optimizer.** There is no performance difference between writing `df.filter(...).groupBy(...)` and writing the equivalent SQL string — choose whichever is more readable for your team.

> **INTERVIEW TIP — PART 17:** "Is SQL slower than the DataFrame API in Spark?" — No. Both go through Catalyst and produce the same physical plan for equivalent logic. Performance differences, if any, come from how the query itself is written, not which syntax (SQL vs DataFrame API) was used.

PART 18 OF 24

Complete PySpark Tutorial

This part is a hands-on, code-first reference covering the operations you will use daily as a PySpark data engineer.

18.1 Creating a SparkSession

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("PySparkTutorial") \
    .config("spark.sql.shuffle.partitions", "50") \
    .getOrCreate()
```

18.2 Reading Data — CSV, JSON, Parquet, Delta

```
# CSV
df_csv = spark.read.option("header", True).option("inferSchema", True).csv("path/data.csv")

# JSON
df_json = spark.read.json("path/data.json")

# Parquet (columnar, schema built-in — the standard for big data)
df_parquet = spark.read.parquet("path/data.parquet")

# Delta Lake (adds ACID transactions and time travel on top of Parquet)
df_delta = spark.read.format("delta").load("path/delta_table")
```

18.3 Writing Data

```
df.write.mode("overwrite").parquet("output/path")
df.write.mode("append").partitionBy("year", "month").parquet("output/path")
df.write.format("delta").mode("overwrite").save("output/delta_table")
```

18.4 Filtering & Sorting

```
df.filter(df.amount > 500).show()
df.where("status = 'ACTIVE' AND amount > 500").show()
df.orderBy(df.amount.desc()).show()
df.sort("customer_id", "amount", ascending=[True, False]).show()
```

18.5 Aggregation

```
from pyspark.sql import functions as F

df.groupBy("customer_id").agg(
    F.sum("amount").alias("total"),
    F.avg("amount").alias("avg_amount"),
    F.count("*").alias("txn_count")
).show()
```

18.6 Joins

```
df_joined = orders.join(customers, on="customer_id", how="inner")
df_joined = orders.join(customers, on="customer_id", how="left")

# Broadcast join hint — for small lookup tables
from pyspark.sql.functions import broadcast
df_joined = orders.join(broadcast(customers), on="customer_id")
```

Join type	`how` value
Inner	`"inner"`
Left Outer	`"left"` / `"left_outer"`
Right Outer	`"right"` / `"right_outer"`
Full Outer	`"outer"` / `"full"`
Left Semi (filter only)	`"left_semi"`
Left Anti (exclude matches)	`"left_anti"`

18.7 Window Functions

```
from pyspark.sql.window import Window
from pyspark.sql import functions as F

w = Window.partitionBy("customer_id").orderBy(F.col("txn_date").desc())

df.withColumn("rank", F.rank().over(w)) \
  .withColumn("running_total", F.sum("amount").over(w.rowsBetween(Window.unboundedPreceding, 0))) \
  .show()
```

18.8 UDF and Pandas UDF

```
# Plain Python UDF — row-by-row, slower (JVM <-> Python serialization per row)
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

@udf(returnType=StringType())
def classify(amount):
    return "HIGH" if amount > 1000 else "LOW"

df.withColumn("tier", classify(df.amount)).show()

# Pandas UDF — vectorized via Apache Arrow, much faster for heavy row-level logic
import pandas as pd
from pyspark.sql.functions import pandas_udf

@pandas_udf(StringType())
def classify_vectorized(amount: pd.Series) -> pd.Series:
    return amount.apply(lambda x: "HIGH" if x > 1000 else "LOW")

df.withColumn("tier", classify_vectorized(df.amount)).show()
```

18.9 Broadcast Variables & Accumulators

```
# Broadcast variable — ship a read-only lookup value to every executor once, not per task
lookup = {"US": "United States", "IN": "India"}
bc = spark.sparkContext.broadcast(lookup)

df.rdd.map(lambda row: bc.value.get(row.country_code)).collect()

# Accumulator — a write-only shared counter that executors can add to, driver can read
error_count = spark.sparkContext.accumulator(0)

def check(row):
    if row.amount is None:
        error_count.add(1)

df.foreach(check)
print(error_count.value)
```

18.10 Caching, Persist, Checkpoint

```
df.cache()                                # persist in memory (and disk if needed)
df.persist(StorageLevel.MEMORY_AND_DISK)    # explicit storage level

spark.sparkContext.setCheckpointDir("hdfs:///checkpoints/")
df.checkpoint()   # truncates lineage by writing to reliable storage — useful for very long DAGs
```

18.11 Sampling & randomSplit

```
sample_df = df.sample(withReplacement=False, fraction=0.1, seed=42)
train_df, test_df = df.randomSplit([0.8, 0.2], seed=42)
```

18.12 explode(), Arrays, Structs, MapType, Nested JSON

```
from pyspark.sql.functions import explode, col

# Nested JSON often has arrays and structs
# {"customer": {"id": 1, "name": "Ravi"}, "items": [{"sku": "A1", "qty": 2}, {"sku": "B2", "qty":
1}]}

df_nested = spark.read.json("orders_nested.json")

df_flat = df_nested \
    .withColumn("item", explode(col("items"))) \
    .select(
        col("customer.id").alias("customer_id"),
        col("customer.name").alias("customer_name"),
        col("item.sku").alias("sku"),
        col("item.qty").alias("qty")
    )
df_flat.show()
```

Type	Use case
ArrayType	A column holding a list of values (e.g., list of tags) — use `explode()` to flatten to rows
StructType	A column holding nested fields (like a mini sub-table) — access with dot notation
MapType	A column holding key-value pairs — access with `col("map_col")["key"]`

INTERVIEW TIP — PART 18: "When would you choose a Pandas UDF over a plain Python UDF?" — Whenever the per-row logic is heavy and can be vectorized (numeric transforms, string parsing at scale) — Pandas UDFs batch rows through Apache Arrow, cutting serialization overhead dramatically compared to row-at-a-time Python UDFs.

Spark Performance Optimization

19.1 The Optimization Toolbox

Technique	What it solves
Caching / Persist	Avoid recomputing a DataFrame that is reused multiple times downstream
Broadcast Join	Avoid shuffling a large table when joined with a small one
Partition Pruning	Skip reading entire partitions (e.g., by date folder) that a filter excludes
Predicate Pushdown	Push filters into the file reader, reading less data off disk in the first place
Bucketing	Pre-shuffle and pre-sort data at write time to speed up repeated joins/ aggregations
Salting	Spread a skewed key across many partitions by adding a random suffix
Avoiding unnecessary shuffles	Combine operations, filter early, minimize `repartition()` calls
Adaptive Query Execution (AQE)	Runtime re-optimization based on actual data statistics
Dynamic Partition Pruning	Prune partitions on the fact table based on a filter applied to a joined dimension table
Skew Join Optimization	Automatically splits oversized partitions in a join (AQE, Spark 3+)
Small Files Problem	Solved via `coalesce()` before write, or compaction jobs

19.2 Adaptive Query Execution (AQE) — Deep Dive

AQE (Spark 3.0+, on by default in modern versions) re-optimizes the query plan **during** execution using real statistics gathered from completed stages — something a purely static, plan-ahead-of-time optimizer cannot do. Its three headline features:

- **Dynamically coalescing shuffle partitions** — merges many small post-shuffle partitions into fewer, right-sized ones.
- **Dynamically switching join strategies** — can switch a sort-merge join to a broadcast join mid-plan if a table turns out smaller than estimated.
- **Dynamically optimizing skew joins** — detects an oversized partition and splits it into smaller sub-tasks automatically.

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
```

19.3 Dynamic Partition Pruning (DPP)

Classic scenario: a huge `fact_sales` table partitioned by date, joined with a small `dim_date` table filtered to a specific quarter. Without DPP, Spark would scan every partition of `fact_sales`. With DPP, Spark first evaluates the filter on the small dimension table, then uses those resulting values to prune (skip) irrelevant partitions of the large fact table before scanning it.

19.4 Bucketing vs Partitioning

Aspect	Partitioning	Bucketing
Mechanism	Splits data into separate folders by column value	Splits data into a fixed number of files, hashed by column value
Best for	Low-cardinality columns used in filters (e.g., date, country)	High-cardinality columns used in joins/ aggregations (e.g., user_id)
Benefit	Partition pruning — skip whole folders	Avoids shuffle on repeated joins on the bucketed column

19.5 Optimization Checklist

- ☐ Use DataFrame/SQL API over RDDs wherever possible
- ☐ Use Parquet/Delta (columnar) instead of CSV/JSON for large datasets
- ☐ Filter and select only needed columns as early as possible
- ☐ Broadcast small tables in joins
- ☐ Enable AQE
- ☐ Cache DataFrames that are reused multiple times — and `unpersist()` when done
- ☐ Watch for and address data skew
- ☐ Right-size `spark.sql.shuffle.partitions` for your data volume
- ☐ Avoid Python UDFs when a built-in function exists; prefer Pandas UDFs otherwise
- ☐ Coalesce before writing to avoid the small-files problem

> **INTERVIEW TIP — PART 19:** "Your job runs fine on sample data but is very slow / fails in production" — This classic scenario question is almost always about **data skew** at production scale, or the default 200 shuffle partitions being wrong for the actual data volume. Always mention checking the Spark UI's stage view for a single task taking far longer than the rest.

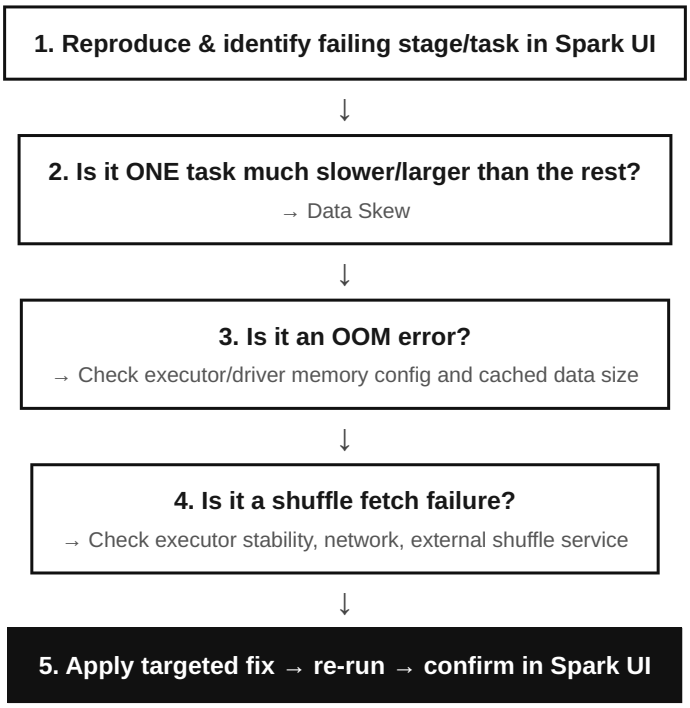
Error Handling & Troubleshooting

20.1 Common Spark Errors

Error	Typical cause	Fix direction
OutOfMemoryError (Executor)	Too little executor memory for the partition size, or data skew	Increase executor memory, repartition, address skew, reduce cached data
OutOfMemoryError (Driver)	`collect()` on a huge dataset, or broadcasting a table too large for the broadcast threshold	Avoid `collect()`, raise/adjust broadcast threshold appropriately, aggregate on executors first
Executor Lost	OOM kill, node failure, or preemption in a shared cluster	Check executor logs; tune memory; enable dynamic allocation with retries
Driver Lost	Driver OOM, network partition, or cluster manager killing the driver	Reduce driver-side collection; increase driver memory if legitimately needed
Task Failed (retried)	Transient node issue, bad record causing a parse error	Spark retries automatically (default 4 attempts); investigate if all retries fail
FetchFailedException (shuffle)	An executor holding shuffle data died or became unreachable	Enable external shuffle service; investigate node stability
GC Overhead Limit Exceeded	Excessive garbage collection, usually too many small objects (heavy RDD usage)	Prefer DataFrames (Tungsten), tune JVM GC settings, increase memory
Serialization Errors	A non-serializable object referenced inside a closure/UDF	Avoid referencing driver-only objects (e.g., open DB connections) inside distributed functions
Network Timeout	Slow shuffle fetch, overloaded network, or an unresponsive executor	Increase timeout configs; investigate cluster network health and skew

20.2 Troubleshooting Flow

GENERAL SPARK TROUBLESHOOTING FLOW



INTERVIEW TIP — PART 20: "Walk me through how you'd debug a slow Spark job in production." — Strong answers always start with the **Spark UI**: identify the slowest stage, check for skewed task durations, check shuffle read/write volumes, check for spill, and only then move to config changes — guessing at configs without profiling first is a weak answer.

100 Spark & PySpark Interview Questions

Organized by category for targeted revision. Use these alongside the detailed explanations in Parts 1–20.

A. Beginner (1–15)

1. What is Apache Spark, in one sentence?
2. What is the difference between Spark and Hadoop MapReduce?
3. What are the main components of the Spark ecosystem?
4. What is a SparkSession, and how is it different from SparkContext?
5. What is an RDD?
6. What is a DataFrame, and how is it different from an RDD?
7. What is the difference between a transformation and an action?
8. Name three transformations and three actions.
9. What is lazy evaluation, and why does Spark use it?
10. What is a partition in Spark?
11. What programming languages does Spark support?
12. What is the difference between `cache()` and `persist()`?
13. What file formats does Spark commonly read and write?
14. What is the role of the Driver program?
15. What is an Executor?

B. Intermediate (16–35)

16. What is the difference between narrow and wide transformations?
17. Explain what a shuffle is and why it is expensive.
18. What is the difference between `repartition()` and `coalesce()`?
19. What is the difference between `groupByKey()` and `reduceByKey()`?
20. What is a broadcast join, and when should you use one?
21. What is the Catalyst Optimizer?
22. What is the Tungsten engine?
23. What is Whole-Stage Code Generation?
24. What is the difference between a Job, a Stage, and a Task?
25. What triggers the creation of a new stage?
26. What is data skew, and how do you detect it?
27. What is the difference between Spark SQL and the DataFrame API in terms of performance?
28. What is a Temporary View vs a Global Temporary View?
29. What is the difference between a Python UDF and a Pandas UDF?
30. What is Py4J, and why does PySpark need it?
31. What is a broadcast variable, and how is it different from an accumulator?
32. What is checkpointing, and when would you use it?

33. What is the difference between DISK_ONLY and MEMORY_AND_DISK storage levels?
34. What is a window function, and give an example use case.
35. What is the difference between `explode()` and `flatten` operations on arrays?

C. Advanced (36–55)

36. Explain the four phases of the Catalyst Optimizer in detail.
37. How does Whole-Stage Code Generation improve CPU efficiency?
38. Explain Unified Memory Management and how storage/execution memory interact.
39. How does Spark achieve fault tolerance without replicating all data?
40. What is Adaptive Query Execution (AQE), and what three optimizations does it enable?
41. What is Dynamic Partition Pruning, and when does it activate?
42. How does the DAG Scheduler decide stage boundaries?
43. How does the Task Scheduler use data locality when assigning tasks?
44. Explain how shuffle write and shuffle read work internally.
45. What is the External Shuffle Service, and why is it important for dynamic allocation?
46. How does Kryo serialization improve performance over Java serialization?
47. What causes spill to disk, and how do you diagnose it in the Spark UI?
48. How does bucketing avoid shuffles on repeated joins?
49. Explain salting as a technique to fix data skew in a join.
50. How does Spark's off-heap (Tungsten) memory reduce garbage collection pauses?
51. What is the difference between a sort-merge join and a broadcast hash join internally?
52. How does AQE dynamically switch join strategies mid-query?
53. What is predicate pushdown, and how does it interact with a columnar format like Parquet?
54. Explain how Spark handles speculative execution of slow tasks.
55. What is the difference between logical and physical query plans?

D. Architecture (56–70)

56. Draw and explain the end-to-end Spark architecture from user code to result.
57. What is the role of the Cluster Manager, and name the four types Spark supports.
58. Compare Standalone, YARN, Kubernetes, and Mesos as cluster managers.
59. What is the Block Manager, and what does it store?
60. How does the Driver communicate with Executors during execution?
61. What happens if the Driver process crashes mid-job?
62. What happens if one Executor crashes mid-job?
63. How does Spark decide how many executors to launch?
64. What is dynamic resource allocation, and how does it work?
65. How is a Spark application different from a Spark job, stage, and task, architecturally?
66. How does Spark run on Kubernetes differently from YARN?
67. What is the relationship between cores, threads, and tasks inside an executor?
68. How does Spark achieve data locality, and why does it matter for performance?

69. What is the difference between client mode and cluster deploy mode?
70. How does the Spark UI help you understand the architecture of a running job?

E. Optimization (71–85)

71. How would you optimize a Spark job that is running slower than expected?
72. How do you decide the right number of shuffle partitions?
73. How do you reduce the small-files problem when writing output?
74. What configuration would you check first for an OOM error?
75. How would you optimize a join between a 1 TB table and a 10 MB table?
76. How would you optimize a join between two very large, skewed tables?
77. Why should you avoid Python UDFs when a built-in function exists?
78. How does caching a DataFrame improve performance, and when can it hurt performance?
79. What is the impact of file format choice (CSV vs Parquet) on performance?
80. How does partition pruning reduce the amount of data scanned?
81. How would you tune executor memory and executor cores for a given cluster?
82. What is the trade-off between more executors with fewer cores vs fewer executors with more cores?
83. How does enabling AQE change your tuning approach?
84. How would you reduce shuffle volume in a complex multi-join pipeline?
85. What role does column pruning play in reading large Parquet datasets?

F. Scenario-Based (86–95)

86. Your job works on 1 GB of sample data but fails on 500 GB of production data — what do you check first?
87. One task out of 200 is taking 40 minutes while the rest finish in 30 seconds — diagnose this.
88. Your driver crashes with an OutOfMemoryError right after calling `.collect()` — what's wrong and how do you fix it?
89. Your output folder has 10,000 tiny files after a write — what caused it and how do you fix it?
90. A join that used to take 2 minutes now takes 40 minutes after a data volume increase — what do you investigate?
91. Your Spark Streaming job is falling behind and building up a backlog — what are the likely causes?
92. A colleague's RDD-based pipeline is much slower than an equivalent DataFrame pipeline — explain why.
93. Your job intermittently fails with FetchFailedException — what does this indicate, and how do you address it?
94. You need to process a dataset that doesn't fit in the cluster's total memory — how do you approach it?
95. Your cached DataFrame doesn't seem to speed up a second action on it — what could be wrong?

G. Production Support (96–100)

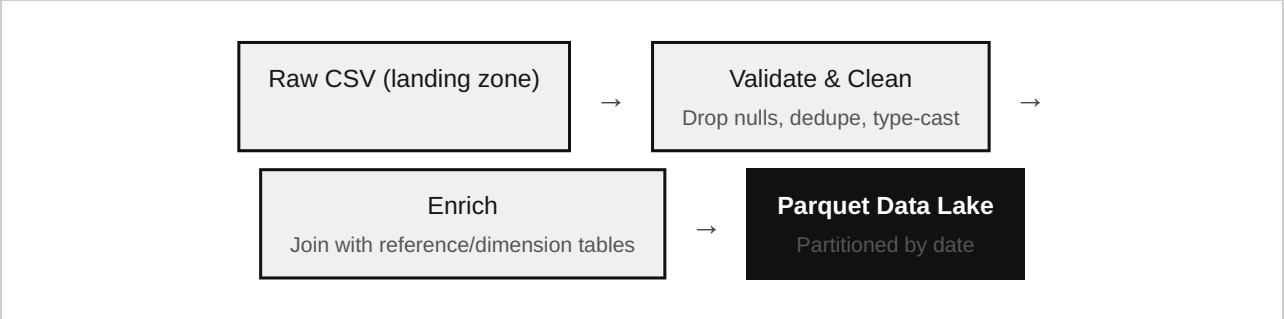
96. How do you monitor a long-running Spark job in production?
97. What metrics in the Spark UI would you check daily for a critical pipeline?
98. How do you set up alerting for Spark job failures in a production pipeline?
99. How do you handle schema evolution in a production pipeline reading from an upstream source?
100. What is your approach to safely re-running a failed production Spark job without duplicating data?

HOW TO USE THIS LIST: Don't just memorize answers — for every question, be ready to give a real example from a project you've worked on. Interviewers consistently rate candidates higher when a conceptual answer is immediately followed by "...and in a project I worked on, this looked like...".

Hands-on Projects

Project 1 — Batch ETL Pipeline

Goal: Ingest raw CSV transaction files, clean and validate them, and load them into a partitioned Parquet data lake.



Key skills practiced: schema enforcement, `dropDuplicates()`, broadcast joins for reference data, partitioned writes, idempotent re-runs (overwrite by partition).

Project 2 — Retail Analytics

Goal: Build customer and product analytics on top of sales data — total spend per customer, top products per region, month-over-month growth.

Key skills practiced: `groupBy` / `agg`, window functions for running totals and rankings, pivoting data, writing results to a BI-friendly Parquet/Delta table.

Project 3 — Log Processing

Goal: Parse raw, semi-structured application/web server logs, extract structured fields (timestamp, status code, endpoint, latency), and compute error-rate dashboards.

Key skills practiced: regex-based parsing with `regexp_extract`, handling malformed/unstructured lines gracefully, aggregating by time window, detecting anomalies (spikes in 5xx errors).

Project 4 — Streaming Pipeline

Goal: Consume a stream of events (e.g., from Kafka), apply transformations, and write continuously updated aggregates.

```
stream_df = (
    spark.readStream
         .format("kafka")
         .option("kafka.bootstrap.servers", "broker:9092")
         .option("subscribe", "transactions")
         .load()
)

parsed = stream_df.selectExpr("CAST(value AS STRING)")
# ... parse JSON, apply windowed aggregation ...

query = parsed.writeStream \
    .outputMode("append") \
    .format("parquet") \
    .option("checkpointLocation", "chk/") \
    .start("output/streaming_result")

query.awaitTermination()
```

Key skills practiced: Structured Streaming's micro-batch model, watermarking for late data, checkpointing for exactly-once-style recovery, output modes (append/update/complete).

Cheat Sheet

23.1 Architecture at a Glance

Driver (plans) → Cluster Manager (allocates) → Executors (run Tasks) → Tasks operate on Partitions → Results return to Driver.

23.2 Command Quick Reference

Need to...	Command
Read Parquet	`spark.read.parquet(path)`
Filter rows	`df.filter(cond)` / `df.where(cond)`
Select/derive columns	`df.select(...)`, `df.withColumn(...)`
Aggregate	`df.groupBy(...).agg(...)`
Join	`df1.join(df2, on=..., how=...)`
Broadcast small table	`df1.join(broadcast(df2), ...)`
Increase parallelism	`df.repartition(n)`
Reduce output files	`df.coalesce(n)`
Cache for reuse	`df.cache()` / `df.persist(level)`
Write output	`df.write.mode("overwrite").parquet(path)`
Run SQL	`df.createOrReplaceTempView("t")` then `spark.sql("...")`

23.3 Optimization Cheat Sheet

- Small table in a join? → **Broadcast it.**
- One task much slower than others? → **Data skew — salt the key or enable AQE skew join.**
- Too many tiny output files? → **Coalesce before writing.**
- DataFrame reused 3+ times downstream? → **Cache it, unpersist when done.**
- Slow row-by-row Python logic? → **Rewrite as a Pandas UDF or built-in function.**
- Job spilling to disk? → **Increase memory, fix skew, or reduce partition size.**

23.4 Interview Tips — Quick Recall

- Spark is faster than MapReduce because of: in-memory computation + DAG-based multi-step optimization + Catalyst/Tungsten code generation.
- Narrow = no shuffle. Wide = shuffle = new stage.

- DataFrames beat RDDs because of Catalyst optimization + Tungsten's compact binary format.
- Always mention checking the **Spark UI** first in any troubleshooting answer.

Final Revision Notes

24.1 The Story of Spark, in One Paragraph

Spark exists because Hadoop MapReduce wrote everything to disk between steps, which was too slow for iterative and interactive workloads. Spark's answer was to keep data in distributed memory and represent computation as a DAG rather than a rigid two-stage Map/Reduce model. A Driver plans the work; a Cluster Manager (Standalone, YARN, Kubernetes, or Mesos) allocates Executors on Worker Nodes; the Driver splits the DAG into Stages at every shuffle boundary, and Stages into Tasks — one Task per Partition. Narrow transformations pipeline cheaply within a stage; wide transformations force an expensive shuffle and a new stage. For structured data, DataFrames (not raw RDDs) let the Catalyst Optimizer rewrite your query into an efficient physical plan, and the Tungsten engine executes that plan using a compact binary memory format and generated, fused JVM code for speed. Lazy evaluation means none of this runs until an Action is called. Getting good production performance is mostly about managing four things well: shuffle volume, data skew, memory pressure, and file/partition sizing — with Adaptive Query Execution increasingly handling much of this automatically in modern Spark.

24.2 The 12 Things to Never Forget Before an Interview

1. Transformations are lazy; Actions trigger execution.
2. Narrow transformations stay in a stage; wide transformations create a shuffle and a new stage.
3. One Job = one Action. Stages = shuffle boundaries + 1. Tasks = partitions per stage.
4. The Driver plans and coordinates; Executors do the actual computation.
5. DataFrames are optimized by Catalyst; raw RDDs are not.
6. Catalyst decides the plan; Tungsten executes it efficiently at the CPU/memory level.
7. Shuffle = network + disk I/O + serialization — always the most expensive operation to watch for.
8. Broadcast joins avoid shuffling the large side of a join when the other side is small.
9. Data skew shows up as one dramatically slower task — fix with salting or AQE.
10. Cache DataFrames that are reused multiple times; unpersist when finished.
11. Prefer built-in functions and Pandas UDFs over plain row-at-a-time Python UDFs.
12. Always start troubleshooting from the Spark UI, not from guessing at configs.

24.3 Closing Note

This guide is designed to be revisited — read it once end-to-end for the full picture, then use Part 21 (Interview Questions) and Part 23 (Cheat Sheet) for rapid pre-interview revision. Mastery of Spark comes from pairing this conceptual foundation with hands-on practice: build the four projects in Part 22 with real (or realistic sample) data, break them intentionally, and use the Spark UI to observe exactly what this guide describes actually happening.

Compiled by Ponugoti Thanush — Data Engineer

A self-study and interview preparation resource on Apache Spark & PySpark